ClearSpend

Data Engineering and Data Governance | EBC 2199

GitHub Repository:
https://github.com/pjgiordan/ClearSpend_DataPlatform/tree/main

Joel Odili-Lock (i6272212)
Paul Carl Giordanelli (i6350376)
Marnix Heutz (i6326166)

1. Introduction

ClearSpend processes millions of card transactions across the United States, partnering with banks and merchants to provide spending and performance analytics. Their infrastructure is fragmented: transactions sit in raw export files, customer and card data is in separate systems, and the merchant category reference file is inconsistent. The finance team cannot reliably report monthly revenue and the analytics team spends most of their time cleaning data rather than analysing it.

This project builds a complete data pipeline using Python and PostgreSQL that takes four raw source files through ingestion, transformation, and a curated warehouse, producing business specific data mart tables that directly answer the questions from all three business teams.

1.1 Technologies Used

- Python with pandas and psycopg2 for data loading and transformation
- PostgreSQL as the database and warehouse
- SQLAlchemy for connecting pandas to PostgreSQL
- SQL (CTAS and INSERT SELECT) for large scale operations

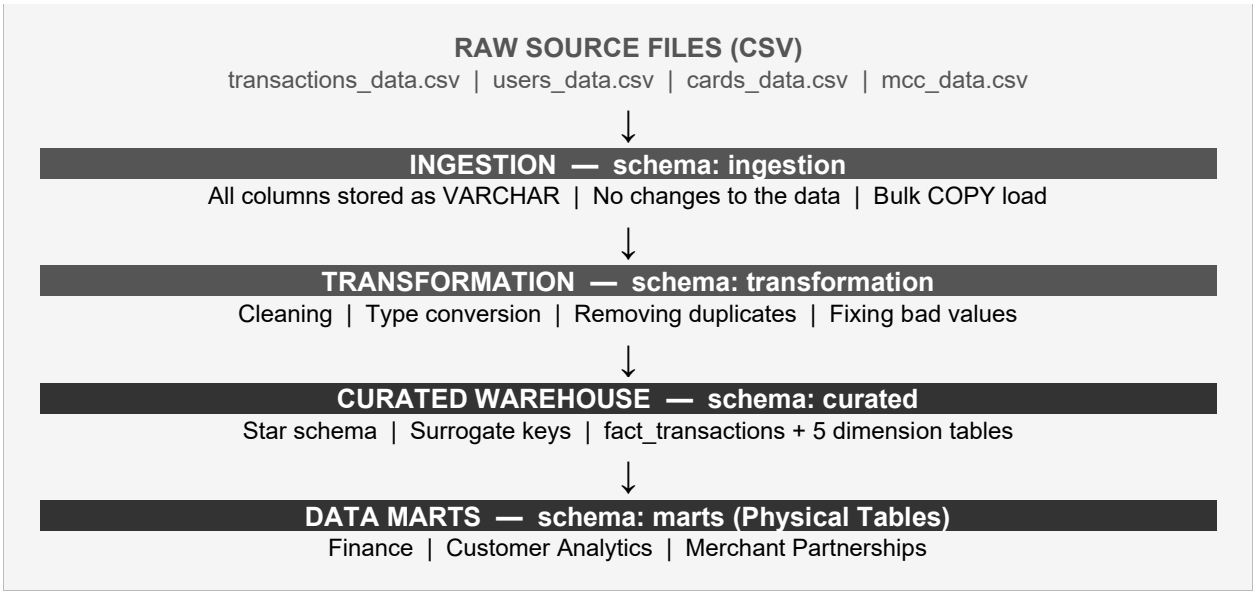
1.2 Source Datasets

Dataset	Number Of Rows	Description
transaction_data	~13.3 million	Card swipes and online purchases
users_data	~2,000	Customer demographic and financial information
cards_data	~6,000	Payment cards linked to customers
mcc_data	127	Merchant Category Code reference file

2. Architecture and Design

2.1 Overview

The platform is split into four PostgreSQL schemas: ingestion, transformation, curated, and marts. Data only moves forward from one layer to the next.



2.2 Technology Choices

PostgreSQL was chosen for its COPY command which bulk loads large CSVs efficiently. Python with pandas handled the cleaning scripts. For the transactions table (13.3 million rows), pure SQL was used instead of pandas to avoid memory constraints. The mart layer uses physical tables rather than views so results are pre-computed and queries do not need to re-scan the full fact table every time.

3. Ingestion Layer

The ingestion layer loads the raw CSV files into PostgreSQL exactly as they arrive, without any changes. This preserves the original data so that if something goes wrong later, it can be referenced. Two scripts handle this: ingestion_DDL.py creates the database and tables, and ingestion_load.py loads the data.

3.1 ingestion_DDL.py

The script uses two connection blocks: first connecting to the default postgres database to create clearspend (if it does not already exist), then reconnecting directly to clearspend to build the schemas and tables. autocommit is set to True because PostgreSQL does not allow CREATE DATABASE inside a transaction. All columns are defined as VARCHAR so the load never fails on messy values like "$1,234.56" — type conversion happens in the transformation stage *(see Appendix A.1)*.

3.2 ingestion_load.py

Each table is truncated before loading to prevent duplicates on re-runs. Files are loaded using PostgreSQL's COPY command, which is significantly faster than row by row INSERT statements, and a row count is printed after each table as a quick check *(see Appendix A.2)*.

4. Transformation Layer

The raw data contains numerous problems: misspellings, inconsistent formatting, missing values, and wrong data types. Each dataset has its own script that reads from ingestion, applies all fixes, and writes to the transformation schema. The ingestion tables are never modified.

4.1 transactions_data

Because this table has 13.3 million rows, loading it into Python would run out of memory. The entire transformation is done using a single SQL statement that creates the clean table directly from the raw one *(see Appendix A.3)*.

The amount field was stored as a string with dollar signs and commas, for example "$1,234.56". These were stripped and the value cast to a decimal number.

The transaction type field (use_chip) had three different labels for in store transactions: Swipe Transaction, Chip Transaction, and Chip Card Transaction. All three were mapped to a single clean value of 'In-Store', and Online Transaction was mapped to 'Online'.

Zip codes had a decimal suffix in some rows, for example '12345.0'. The decimal part was removed.

NULL values in merchant_state and errors were replaced with 'N/A' so those columns are never empty.

4.2 cards_data

This dataset had the most quality issues of all four files *(see Appendix A.4)*.

Card brand had over 20 different ways of writing the same value, including typos, abbreviations, and casing differences. All variants were mapped to a single canonical value.

Card type had over 30 raw variants including abbreviations like 'CC' and 'DB', typos like 'Debti' and 'Crdeit', and concatenated strings like 'CreditCard'. All were collapsed to three clean categories: Credit, Debit, and Debit (Prepaid).

Credit limits came in formats like '$5,000', '$3k', '2.5m', and sometimes negative. A dedicated parsing function strips the dollar sign and commas, handles the k and m shorthand, and flips negatives to positive.

Card numbers were stored as floats in the raw file, for example 4344676511950444.0. These were converted back to clean integer strings and then deduplicated, keeping the last occurrence.

Expiry date and account open date were stored as text in Mon-YY format, for example 'Dec-22'. These were parsed and reformatted to MM/YYYY.

CVV was stored as a number so leading zeros were lost. Values were zero padded back to 3 digits.

has_chip and card_on_dark_web were stored as the strings 'YES' and 'NO'. Both were converted to proper boolean values since a column with only two possible values should not be stored as text.

Issuer bank state had a mix of full state names and lowercase abbreviations. All were standardised to two letter uppercase codes.

Issuer bank name, type, and risk rating all had casing variants and abbreviations that were mapped to consistent values.

4.3 users_data

Users were deduplicated on their ID keeping the last occurrence, since the last record is assumed to be the most recently updated one *(see Appendix A.5)*.

Current age was recalculated from birth_year and birth_month rather than trusting the stored value, which goes out of date every year.

Income and debt fields had the same formatting issues as credit limits. The same parsing function was reused to handle dollar signs, commas, and k/m shorthand.

Credit score had some clearly invalid values like 9999. Any value outside the valid US range of 300 to 850 was set to NULL.

Addresses were sometimes stored in all uppercase. Words that were fully uppercase were title cased while already mixed case words were left unchanged.

Employment status and education level both had a large number of freetext variations, typos, and abbreviations all representing the same underlying category. Each was mapped to a small set of clean, consistent values.

4.4 mcc_data

The MCC file had structural problems on top of the usual messy values *(see Appendix A.6)*.

The code column contained values with the prefix 'MCC', values wrapped in quotes, and metadata rows with values like 'NOTE' or 'COMMENT' mixed in throughout the file. The prefix and quotes were stripped, and then only rows with a purely numeric code were kept, filtering out all metadata rows.

After cleaning, some duplicate codes remained because different raw variants resolved to the same number. Duplicates were removed keeping the first occurrence.

Descriptions were standardised to title case. The notes column was dropped. The updated_by column is kept in the transformation layer as an audit trail but is intentionally dropped when building the warehouse dimension.

5. Data Warehouse Design

5.1 Structure

The warehouse uses a star schema. There is one central fact table that stores all the transactions, and five dimension tables around it holding the descriptive information about customers, cards, dates, merchants, and merchant categories. The full schema including all columns, data types, and relationships is shown below.

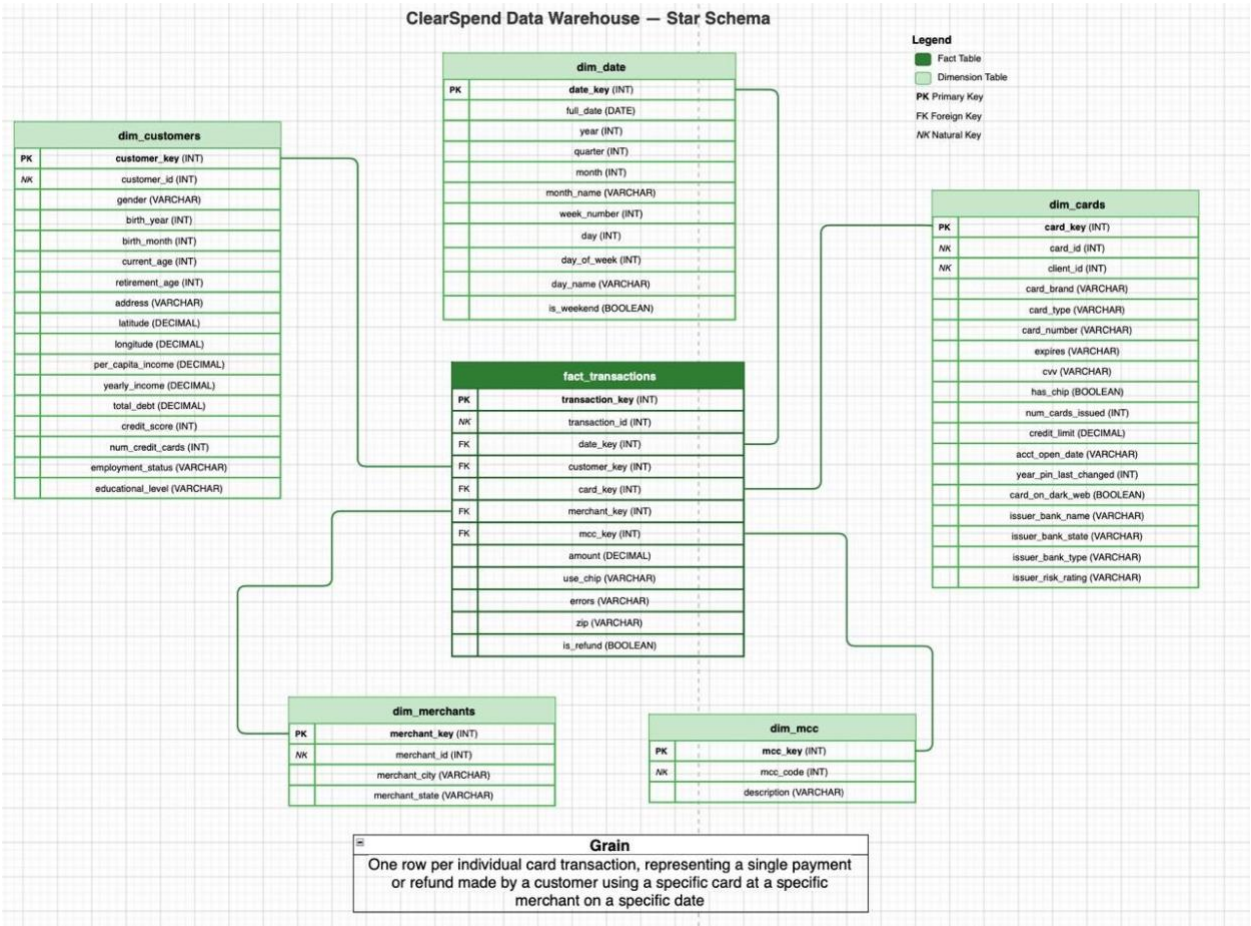


Figure 1: ClearSpend Data Warehouse — Star Schema (draw.io)

5.2 Dimension Tables

dim_customer is built from transformation.users. It holds demographic and financial profile data. The natural key customer_id is kept alongside the surrogate key customer_key.

dim_card is built from transformation.cards. It retains client_id as an extra column so a card can be linked to its owner without going through the fact table. has_chip and card_on_dark_web are stored as booleans.

dim_date is generated from code rather than loaded from a file. The script queries the min and max dates from the transactions table and generates one row per calendar day, with columns for year, quarter, month, week number, day name, and is_weekend stored as a boolean.

dim_merchant is derived from the transactions table since no separate merchants source file was provided. The most frequently occurring city and state combination per merchant is used to produce one clean record per merchant.

dim_mcc holds the MCC code and its standardised description. The updated_by audit field is excluded here as it has no analytical value.

Each dimension is assigned a surrogate key, which is a simple auto-generated integer. The fact table stores these surrogate keys rather than the original source IDs *(see Appendix A.7)*.

5.3 fact_transactions

The fact table stores one row per transaction. The grain is: **one card transaction representing a single payment or refund made by a customer using a specific card at a specific merchant on a specific date** (this is also seen at the bottom of figure 1) . It holds the surrogate keys from all five dimensions alongside the transaction measures: amount, is_refund, use_chip, zip, and errors.

The table is built entirely in SQL using an INSERT SELECT statement that joins the transformation transactions table to all five dimensions. LEFT JOINs are used so that transactions with no matching dimension record are still included rather than being silently dropped. The is_refund flag is derived from the amount: negative amounts are refunds. This means mart queries can filter on is_refund = TRUE without checking the sign of amount each time *(see Appendix A.8)*.

6. Data Marts

The mart layer consists of eleven physical tables across three Python scripts, one per business team. Each table is built using CREATE TABLE AS SELECT directly in PostgreSQL. Because they are stored tables rather than views, queries run much faster since the results are already computed. Each script drops and recreates its tables so they can be refreshed by re-running the script.

6.1 Finance Mart (mart_finance.py)

finance_monthly answers both the monthly revenue question and the refund rate question in a single table. It groups transactions by year and month and returns gross revenue, total refund amount, net revenue, refund count, and refund rate as a percentage. Net revenue is calculated as SUM(amount) directly since refunds are already stored as negative values in the fact table.

finance_by_state shows gross revenue, net revenue, transaction count, and each state's percentage share of total revenue. Online transactions are excluded since they have no physical state. The percentage share uses a window function over the full result set.

finance_by_category joins to dim_mcc and groups by merchant category code. It returns gross revenue, net revenue, transaction count, and average transaction amount per category, ordered by net revenue.

Code reference: *(see Appendix A.9)*

6.2 Customer Analytics Mart (mart_customers.py)

customer_ltv calculates lifetime value per customer. As well as total spend and transaction count, it includes demographic context (gender, age, employment status, education, income, debt, credit score), total amount refunded, average transaction amount, and the date of first and last transaction. Refunds are excluded from lifetime spend.

customer_channel breaks down each customer's transactions by channel. For every customer it returns the number and total spend for online and in store transactions separately, along with the percentage split. This is per customer rather than an overall summary, which is more useful for segmentation.

customer_cards joins dim_card directly to dim_customer via client_id. For each customer it returns total cards, how many have a chip, how many are on the dark web, average and total credit limit, and a count of each card type (credit, debit, prepaid).

suspicious_transactions uses a CTE to calculate per-customer average and standard deviation for non-refund transaction amounts. A transaction is flagged if it meets any of three conditions: the card was linked to a dark web exposure, the transaction has a recorded error, or the amount is more than 3 standard deviations above that customer's own average. Using the customer's own statistics makes the detection personalised rather than applying a single global threshold *(see Appendix A.10)*.

6.3 Merchant Partnerships Mart (mart_merchants.py)

merchant_performance aggregates transaction count, gross revenue, net revenue, refund count, and average transaction amount per merchant. The merchant's category from dim_mcc is included alongside their city and state.

industry_growth uses the same CTE self-join approach as the finance category mart. It stores net revenue per category per year, then joins to the prior year to calculate year over year growth

percentage. NULLIF prevents a division by zero error when a category had no prior year revenue.

merchant_errors calculates error rate per merchant as a percentage of total transactions. It also uses STRING_AGG to collect the distinct error types seen for each merchant into a single column, which makes it easy to see not just how many errors a merchant has but what kinds.

revenue_by_geography goes down to city level rather than just state, grouping by both merchant_state and merchant_city. This gives the merchant partnerships team more granular geographic insight. Online transactions are excluded.

Code reference: *(see Appendix A.11)*

6.4 Business Questions Coverage

Team	Business Question	Table
Finance	Total revenue by month	marts.finance_monthly
Finance	Percentage of transactions that are refunds	marts.finance_monthly
Finance	Which states generate the most revenue	marts.finance_by_state
Finance	Which categories drive the highest spending	marts.finance_by_category
Customer	Lifetime value of each customer	marts.customer_ltv
Customer	Online vs in store behaviour	marts.customer_channel
Customer	How many active cards per customer	marts.customer_cards
Customer	Suspicious transaction patterns	marts.suspicious_transactions
Merchant	Highest transaction volume merchants	marts.merchant_performance
Merchant	Fastest growing industries year over year	marts.industry_growth
Merchant	Merchants with the highest error rates	marts.merchant_errors
Merchant	Geographic revenue distribution	marts.revenue_by_geography

7. Code Quality and Reproducibility

7.1 Project Structure

File	Purpose
data/raw/	Source CSV files (transactions_data.csv excluded from Github, as it exceeds the 100MB file size limit)
ingestion_DDL.py	Creates the database, schema, and raw tables
ingestion_load.py	Loads CSV files into the ingestion schema
transformation_transactions.py	Cleans transactions using SQL CTAS
transformation_cards.py	Cleans the cards table using Python
transformation_users.py	Cleans the users table using Python
transformation_mcc.py	Cleans the MCC reference table using Python
dim_*.py (5 files)	Builds each dimension table in the curated schema
fact_transactions.py	Builds the central fact table
mart_finance.py	3 mart tables for the finance team
mart_customers.py	4 mart tables for the customer analytics team
mart_merchants.py	4 mart tables for the merchant partnerships team
README.md	Setup instructions and run order

7.2 How to Run the Pipeline

- Step 1: Run ingestion_DDL.py to create the database and tables
- Step 2: Run ingestion_load.py to load the CSV files
- Step 3: Run the four transformation scripts (any order)
- Step 4: Run the five dimension scripts (any order)
- Step 5: Run fact_transactions.py to build the fact table
- Step 6: Run mart_finance.py, mart_customers.py, mart_merchants.py

Each script drops its target table before creating it again so the pipeline can be re-run from any point without leaving behind old data.

7.3 Documentation

Every script has a header block stating the layer, source, and target. Key decisions are explained inline throughout the code and row counts are printed after each load as an audit trail. In a production setup, the hardcoded credentials and file paths would be moved to environment variables.

8. Conclusion

This project transformed four raw source files into a structured warehouse with eleven business facing mart tables, covering every stage from ingestion through to answering all twelve business questions. Key decisions include using SQL CTAS for the 13.3 million row transactions table, recalculating age from birth date on every run, deriving the merchant dimension from transaction data, using LEFT JOINs to preserve all transactions, and storing boolean columns as proper booleans throughout.

Future improvements would include externalising credentials to environment variables, adding a scheduler to automate the pipeline, and adding indexes to mart tables for faster filtering.

Appendix: Code Reference

The following code snippets correspond to the numbered references in the report body.

A.1 ingestion_DDL.py — Database creation and two-step connection

```
conn = psycopg2.connect(database='postgres', ...)
conn.autocommit = True
cursor.execute("SELECT 1 FROM pg_database WHERE datname = 'clearspend'")
if not cursor.fetchone():
    cursor.execute("CREATE DATABASE clearspend")
conn.close()
# Second connection directly to clearspend
conn = psycopg2.connect(database='clearspend', ...)
cursor.execute("CREATE SCHEMA IF NOT EXISTS ingestion")
```

A.2 ingestion_load.py — Bulk COPY load

```
cursor.execute(f"TRUNCATE TABLE {table_name}")
with open(file_path, 'r', encoding='utf-8') as f:
    cursor.copy_expert(f"COPY {table_name} FROM STDIN WITH CSV HEADER", f)
cursor.execute(f"SELECT COUNT(*) FROM {table_name}")
count = cursor.fetchone()[0]
print(f"Loaded {count} rows into {table_name}")
```

A.3 transformation_transactions.py — SQL CTAS transformation

```
CREATE TABLE transformation.transactions AS
SELECT
    id::BIGINT,
    date::TIMESTAMP::DATE,
    REPLACE(REPLACE(amount, '$', ''), ',', '')::DECIMAL(18,2) AS amount,
    CASE
        WHEN use_chip = 'Swipe Transaction'     THEN 'In-Store'
        WHEN use_chip = 'Chip Transaction'      THEN 'In-Store'
        WHEN use_chip = 'Chip Card Transaction' THEN 'In-Store'
        WHEN use_chip = 'Online Transaction'    THEN 'Online'
        ELSE 'Unknown'
    END AS use_chip,
    COALESCE(merchant_state, 'N/A') AS merchant_state,
    SPLIT_PART(zip, '.', 1) AS zip,
    COALESCE(errors, 'N/A') AS errors
FROM ingestion.transactions WHERE id IS NOT NULL
```

A.4 transformation_cards.py — Key cleaning logic

```
# Lookup map example
card_brand_map = {'V': 'Visa', 'V!sa': 'Visa', 'VISA': 'Visa', ...}
df['card_brand'] = df['card_brand'].replace(card_brand_map)

# Credit limit parser
def parse_credit_limit(val):
    val = str(val).strip().replace('$', '').replace(',', '')
    if val.lower().endswith('k'): return float(val[:-1]) * 1_000
    if val.lower().endswith('m'): return float(val[:-1]) * 1_000_000
    return abs(float(val))
```

```
# Card number float to string + deduplication
df['card_number'] = df['card_number'].apply(
    lambda x: str(int(float(x))) if pd.notna(x) else None)
df = df.drop_duplicates(subset=['card_number'], keep='last')

# CVV zero padding
df['cvv'] = df['cvv'].apply(
    lambda x: str(int(float(x))).zfill(3) if pd.notna(x) else None)

# Boolean conversion
df['has_chip'] = df['has_chip'].str.upper().map({'YES': True, 'NO': False})
df['card_on_dark_web'] = df['card_on_dark_web'].str.upper().map({'YES': True,
'NO': False})
```

A.5 transformation_users.py — Key cleaning logic

```
df = df.drop_duplicates(subset=['id'], keep='last')

today = pd.Timestamp.today()
df['current_age'] = df.apply(
    lambda row: int(today.year - row['birth_year'] - (
        1 if today.month < row['birth_month'] else 0
    )) if pd.notna(row['birth_year']) else None, axis=1)

df['credit_score'] = pd.to_numeric(df['credit_score'], errors='coerce')
df.loc[(df['credit_score'] < 300) | (df['credit_score'] > 850),
'credit_score'] = None

df['address'] = df['address'].apply(
    lambda x: ' '.join(word.title() if word.isupper() else word
        for word in str(x).split()) if pd.notna(x) else None)
```

A.6 transformation_mcc.py — Key cleaning logic

```
df['code'] = df['code'].str.replace('"', '', regex=False).str.strip()
df['code'] = df['code'].str.replace('MCC', '', case=False,
regex=False).str.strip()
df = df[df['code'].str.match(r'^\d+$', na=False)]
df = df.drop_duplicates(subset=['code'], keep='first')
```

A.7 dim_*.py — Surrogate key generation

```
dim_customer = dim_customer.sort_values('customer_id').reset_index(drop=True)
dim_customer.insert(0, 'customer_key', range(1, len(dim_customer) + 1))

# dim_mcc intentionally excludes updated_by
df = pd.read_sql('SELECT code, description FROM transformation.mcc',
con=engine)

# dim_date is_weekend as boolean
'is_weekend': (date_range.dayofweek >= 5)
```

A.8 fact_transactions.py — Fact table build

```
INSERT INTO curated.fact_transactions (
```

```
    transaction_id, date_key, customer_key, card_key,
    merchant_key, mcc_key, amount, is_refund, use_chip, zip, errors)
SELECT t.id, dd.date_key, dc.customer_key, dca.card_key,
       dm.merchant_key, dmcc.mcc_key, t.amount,
       CASE WHEN t.amount < 0 THEN TRUE ELSE FALSE END AS is_refund,
       t.use_chip, t.zip, t.errors
FROM transformation.transactions t
LEFT JOIN curated.dim_date     dd   ON t.date        = dd.full_date
LEFT JOIN curated.dim_customer dc   ON t.client_id   = dc.customer_id
LEFT JOIN curated.dim_card     dca  ON t.card_id     = dca.card_id
LEFT JOIN curated.dim_merchant dm   ON t.merchant_id = dm.merchant_id
LEFT JOIN curated.dim_mcc      dmcc ON t.mcc         = dmcc.mcc_code
```

A.9 mart_finance.py — Monthly revenue and state revenue

```
-- finance_monthly: net revenue = SUM(amount) since refunds are negative
SELECT d.year, d.month, d.month_name,
    SUM(CASE WHEN f.is_refund = FALSE THEN f.amount ELSE 0 END) AS
gross_revenue,
    SUM(CASE WHEN f.is_refund = TRUE THEN ABS(f.amount) ELSE 0 END) AS
total_refund_amount,
    SUM(f.amount) AS net_revenue,
    ROUND(SUM(CASE WHEN f.is_refund THEN 1 ELSE 0 END) * 100.0
        / NULLIF(COUNT(*), 0), 2) AS refund_rate_pct
FROM curated.fact_transactions f
JOIN curated.dim_date d ON f.date_key = d.date_key
GROUP BY d.year, d.month, d.month_name

-- finance_by_state: percentage share using window function
ROUND(SUM(f.amount) * 100.0 / SUM(SUM(f.amount)) OVER (), 2) AS
pct_of_total_revenue
```

A.10 mart_customers.py — Suspicious transactions with per-customer stats

```
WITH customer_stats AS (
    SELECT customer_key,
           AVG(amount) AS avg_amount, STDDEV(amount) AS std_amount
    FROM curated.fact_transactions WHERE is_refund = FALSE
    GROUP BY customer_key
)
SELECT f.transaction_id, c.customer_id, f.amount,
    CASE
        WHEN dc.card_on_dark_web = TRUE          THEN 'Dark Web Card'
        WHEN f.errors != 'N/A'                   THEN 'Transaction Error'
        WHEN f.amount > cs.avg_amount + 3 * cs.std_amount
                                                 THEN 'High Value Outlier'
        ELSE 'Other'
    END AS flag_reason
FROM curated.fact_transactions f
JOIN curated.dim_card dc    ON f.card_key     = dc.card_key
JOIN curated.dim_customer c ON f.customer_key = c.customer_key
JOIN customer_stats cs      ON f.customer_key = cs.customer_key
WHERE dc.card_on_dark_web = TRUE OR f.errors != 'N/A'
   OR f.amount > cs.avg_amount + 3 * cs.std_amount
```

A.11 mart_merchants.py — Error rates with STRING_AGG and industry growth

```
-- merchant_errors: STRING_AGG collects distinct error types per merchant
STRING_AGG(DISTINCT CASE WHEN f.errors != 'N/A' THEN f.errors END, ', ')
    AS error_types
HAVING SUM(CASE WHEN f.errors != 'N/A' THEN 1 ELSE 0 END) > 0

-- industry_growth: CTE self-join for year over year growth
WITH spending_by_year AS (
    SELECT mc.mcc_code, mc.description AS category, d.year,
           SUM(f.amount) AS net_revenue
    FROM curated.fact_transactions f
    JOIN curated.dim_mcc mc ON f.mcc_key = mc.mcc_key
    JOIN curated.dim_date d ON f.date_key = d.date_key
    GROUP BY mc.mcc_code, mc.description, d.year
)
SELECT curr.category, curr.year, curr.net_revenue,
       ROUND((curr.net_revenue - prev.net_revenue) * 100.0
           / NULLIF(prev.net_revenue, 0), 2) AS yoy_growth_pct
FROM spending_by_year curr
LEFT JOIN spending_by_year prev
    ON curr.mcc_code = prev.mcc_code AND curr.year = prev.year + 1
```

Pipeline Execution Order

#	Script	What it produces
1	**ingestion_DDL.py**	clearspend database + ingestion schema + 4 raw tables
2	**ingestion_load.py**	13.3 million rows loaded across 4 tables
3	**transformation_*.py (x4)**	4 cleaned tables in the transformation schema
4	**dim_*.py (x5)**	5 dimension tables in the curated schema
5	**fact_transactions.py**	13.3 million row fact table in the curated schema
6	**mart_finance.py**	3 mart tables: finance_monthly, finance_by_state, finance_by_category
7	**mart_customers.py**	4 mart tables: customer_ltv, customer_channel, customer_cards, suspicious_transactions
8	**mart_merchants.py**	4 mart tables: merchant_performance, industry_growth, merchant_errors, revenue_by_geography

Schema	Objects	Type	Purpose
ingestion	4 tables	Tables	Raw data, unchanged from source
transformation	4 tables	Tables	Cleaned and properly typed data

curated	6 tables	Tables	Star schema warehouse
marts	11 tables	Tables	Pre-computed business team reporting